Notes:

- HW3, HW4 graded, solutions posted
  - Opened issue on github repo if score on these parts were not 30/30 or 20/20
- HW5 posted, due Nov 18
- HW6 posted

Last class:

- more work on classes (`__add__` , `__radd__`)
- programming style
- PEP8 rules
  - indentation and alignment
  - spaces (and not tabs)
  - naming conventions
- running flake8

Today: debugging

Debugging is a significant task when we write complex code.

There are many different ways that we can write code that "doesn't work"

Python is very good at catching and reporting some of these, VSCode is very good at preventing some others

Syntax errors are when we write code that isn't valid Python code

Lots of possibilities:

- forget a bracket or a quote
- mis-indent a line of code or a region of code
- Use incorrect calling sequence
- many more

In [1]:

```python
for i in range(10):
    print(f"The square of {i+1} is {(i+1**2}")
```

```
  Cell In[1], line 2
    print(f"The square of {i+1} is {(i+1**2}")
                                           ^
SyntaxError: closing parenthesis '}' does not match opening parenthesis '('
```

Had to explicitly remove a parenthesis that was added by VSCode to achieve this, but Python immediately gives an informative error message about the specific syntax error `closing parenthesis does not match opening parenthesis`

Similarly, some other common errors are caught and reported by Python at runtime

In [2]:

```python
import math
my_vals = [1, 0.5, 0, -0.5, -1]
for val in my_vals:
    print(f"sinc({val}) = {math.sin(val)/val}")
```

```
sinc(1) = 0.8414709848078965
sinc(0.5) = 0.958851077208406
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[2], line 4
      2 my_vals = [1, 0.5, 0, -0.5, -1]
      3 for val in my_vals:
----> 4     print(f"sinc({val}) = {math.sin(val)/val}")

ZeroDivisionError: float division by zero
```

Gives `ZeroDivisionError` when `val =0`

Or

In [3]:

```python
math.sin(1.0, math.pi)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[3], line 1
----> 1 math.sin(1.0, math.pi)

TypeError: math.sin() takes exactly one argument (2 given)
```

gives `TypeError` because we called a function with too many arguments

The most difficult errors to catch in Python are what we call *logical errors*, where the code is technically correct (meaning valid Python code), but does the wrong thing.

We use testing to ensure that our code does the right thing, but what tools can we use to understand and fix bugs when those tests fail.

For many bugs, adding temporary print statements to our code is a strong first strategy for pinpointing logical errors.

Consider the following code:

In [6]:

```python
def my_mean(nums):
    it = 0
    sum = 0
    while it < len(nums):
        sum += nums[it]
    return sum/len(nums)

my_list = [1, 2, 3, 4, 5, 6]
my_mean(my_list)
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[6], line 9
      6     return sum/len(nums)
      8 my_list = [1, 2, 3, 4, 5, 6]
----> 9 my_mean(my_list)

Cell In[6], line 4, in my_mean(nums)
      2 it = 0
      3 sum = 0
----> 4 while it < len(nums):
      5     sum += nums[it]
      6 return sum/len(nums)

KeyboardInterrupt:
```

First sign that something is wrong is that the code starts running and doesn't stop!

- Hit "interrupt" or "stop" button, VSCode's way of sending kill signal (Ctrl-C at the command line)

Where is the problem?

Step 1 is to narrow scope

- add print statements to see how far we get in the code before trouble happens.

In [7]:

```python
def my_mean(nums):
    it = 0
    sum = 0
    print("Before while loop")
    while it < len(nums):
        sum += nums[it]
    print("After the while loop")
    return sum/len(nums)

my_list = [1, 2, 3, 4, 5, 6]
my_mean(my_list)
```

```
Before while loop
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[7], line 11
      8     return sum/len(nums)
     10 my_list = [1, 2, 3, 4, 5, 6]
---> 11 my_mean(my_list)

Cell In[7], line 5, in my_mean(nums)
      3 sum = 0
      4 print("Before while loop")
----> 5 while it < len(nums):
      6     sum += nums[it]
      7 print("After the while loop")

KeyboardInterrupt:
```

Confirms that we reach the while loop, but do not get past it.

NOTE: this code is exaggeratedly simple: only one real spot where trouble might arise.

However, what we're doing here is really a type of hypothesis testing. Where do we think the error is? Test that our hypothesis is correct.

Here, use print statements to confirm the hypothesis that the problem is in the while loop...

For more complex codes, can use print statements to identify when an input is not what is expected, or to narrow down which part of a code is not doing what is expected. Think about your test cases here, a good test case can help you accomplish this.

Here, add more print statements, now inside while loop:

In [ ]:

```python
def my_mean(nums):
    it = 0
    sum = 0
    print("Before while loop")
    while it < len(nums):
        print(f"Adding {nums[it]}")
        sum += nums[it]
        print(f"sum is now {sum}")
    print("After the while loop")
    return sum/len(nums)

my_list = [1, 2, 3, 4, 5, 6]
my_mean(my_list)
```

From here, problem is clear: we're always adding the same number, so we're not progressing through the list.

This is because we aren't incrementing `it` within the loop. Fix this:

In [9]:

```python
def my_mean(nums):
    it = 0
    sum = 0
    print("Before while loop")
    while it < len(nums):
        print(f"Adding {nums[it]}")
        sum += nums[it]
        it += 1
        print(f"sum is now {sum}")
    print("After the while loop")
    return sum/len(nums)

my_list = [1, 2, 3, 4, 5, 6]
my_mean(my_list)
```

```
Before while loop
Adding 1
sum is now 1
Adding 2
sum is now 3
Adding 3
sum is now 6
Adding 4
sum is now 10
Adding 5
sum is now 15
Adding 6
sum is now 21
After the while loop
```

Out[9]:

```
3.5
```

Now code runs and does what is expected.

Having confirmed this, we can now remove our print statements and return to "clean code"

In [10]:

```python
def my_mean(nums):
    it = 0
    sum = 0
    while it < len(nums):
        sum += nums[it]
        it += 1
    return sum/len(nums)

my_list = [1, 2, 3, 4, 5, 6]
my_mean(my_list)
```

Out[10]:

```
3.5
```

Check again that code runs as expected once we've removed them!

Another tool for debugging is the graphical debugger in VSCode

To run this, click on "debug" icon on left-hand side, to open the debug panel. Then select "debug cell" from beside cell with code

In [12]:

```python
def my_mean(nums):
    it = 0
    sum = 0
    while it < len(nums):
        sum += nums[it]
    return sum/len(nums)

my_list = [1, 2, 3, 4, 5, 6]
my_mean(my_list)
```

```
---------------------------------------------------------------------------
KeyboardInterrupt                         Traceback (most recent call last)
Cell In[12], line 9
      6     return sum/len(nums)
      8 my_list = [1, 2, 3, 4, 5, 6]
----> 9 my_mean(my_list)

Cell In[12], line 4, in my_mean(nums)
      2 it = 0
      3 sum = 0
----> 4 while it < len(nums):
      5     sum += nums[it]
      6 return sum/len(nums)

KeyboardInterrupt:
```

At first, doesn't seem this does anything useful. Notice that we have a new control panel...

Several options in this panel. When code is running, hit "pause" button. See one line highlighted and several other options:

- Continue (just continues execution)
- Step over and step into. These both take one step in the code and pause again. "step over" tries to complete execution regardless of next line, even if it calls another function. "step into" will step to next executed line of code, even if it is in a different function.
- "step out" is counterpart to "step into", and will try to complete execution of current function.
- "restart" to restart from the beginning
- "disconnect" to end debugging.

These aren't all super useful until we add a "breakpoint" to our code. Do this by clicking in left-hand margin in VSCode, next to line where we want to stop execution.

Do this next to line 5 in the code and "hit play". Now code should stop each time we hit the breakpoint

Top-left panel shows us values of variables within our functions. By stepping through, either with "continue" (which continues until it next encounters a breakpoint) or using "step over", we see how variables change. This is equally effective in tracking down the bug in this routine.

Third way to debug is using a command-line debugger like `pdb`.

Open code in a standalone python file. At command line:

```
(CMSC6950_venv) [math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python debug_test.py
^CTraceback (most recent call last):
  File "/Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py", line 10, in <module>
    print(my_mean(my_list))
          ~~~~~~~^^^^^^^^^
  File "/Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py", line 5, in my_mean
    while it < len(nums):
          ^^^^^^^^^^^^^^
KeyboardInterrupt
```

Just runs, no output. Hit Ctrl-C to cancel.

Go to Python code, add `import pdb` and `pdb.set_trace()` before while loop:

Now run at command line:

Now stops with `(Pdb)` prompt. Can type `step` or `s` to take steps (this is the same as `step into` in the graphical debugger; `next` or `n` is the same as `step over`). Alternately, hit `q` to quit

```
(CMSC6950_venv) [math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python debug_test.py
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(7)my_mean()
-> pdb.set_trace()
(Pdb) s
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(8)my_mean()
-> while it < len(nums):
(Pdb) s
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(9)my_mean()
-> sum += nums[it]
(Pdb) s
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(8)my_mean()
-> while it < len(nums):
(Pdb) s
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(9)my_mean()
-> sum += nums[it]
(Pdb) s
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(8)my_mean()
-> while it < len(nums):
(Pdb) s
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(9)my_mean()
-> sum += nums[it]
(Pdb) s
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(8)my_mean()
-> while it < len(nums):
(Pdb) s
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(9)my_mean()
-> sum += nums[it]
(Pdb) s
> /Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py(8)my_mean()
-> while it < len(nums):
(Pdb) q
Traceback (most recent call last):
  File "/Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py", line 14, in <module>
    print(my_mean(my_list))
          ~~~~~~~^^^^^^^^^
  File "/Users/smaclachlan/Work/teaching/CMSC6950/Fall2025/Notes/debug_test.py", line 8, in my_mean
    while it < len(nums):
          ^^
  File "/opt/homebrew/Cellar/python@3.13/3.13.3_1/Frameworks/Python.framework/Versions/3.13/lib/python3.13/bdb.py", line 102, in trace_dispatch
    return self.dispatch_line(frame)
           ~~~~~~~~~~~~~~~~~~^^^^^^^
  File "/opt/homebrew/Cellar/python@3.13/3.13.3_1/Frameworks/Python.framework/Versions/3.13/lib/python3.13/bdb.py", line 129, in dispatch_line
    if self.quitting: raise BdbQuit
                      ^^^^^^^^^^^^^
bdb.BdbQuit
(CMSC6950_venv) [math-scott02:CMSC6950/Fall2025/Notes] smaclachlan%
```

Can add a breakpoint to the code and run again (don't even need to import pdb to add a breakpoint)

Now, can use `c` to continue to next breakpoint

This is only useful if we can also examine variable values, which we can do with `p sum` at the `(Pdb)` prompt.

Using pdb in this manner is helpful in 2 settings:

1. When working with code on a remote server, where tools like VSCode are not easily accessible (e.g., Digital Research Alliance of Canada servers)
2. When we have tests that are failing. Can get pytest to call debugger when a test fails by running

```
pytest --pdb -s -x test_my_stuff.py
```

where `--pdb` says to run pdb on failure, `-s` tells pytest to print all output (to help understand the failed test) and `-x` tells pytest to quit and call pdb at first failed test

However we invoke these tools, our general pattern for debugging is:

1. Observe an unexpected result
2. Use debugger to find first place that something goes wrong with simple inputs that reproduce unexpected results (minimum failing example == MFE).
3. Fix code at this point
4. Repeat until we see expected result

Again, view this from perspective of hypothesis testing:

Always ask "what *should* this code do?" And test on a data set where you can express intermediate answers exactly.

Use this to refine hypothesis about what is wrong in your code. Use print statements or debugger to identify where variables take unexpected values to help refine ideas on what might be incorrect.

Consider another example:

In [13]:

```python
def compute_grade(percent):
    if percent > 80:
        return "A"
    elif percent > 55:
        return "C"
    elif percent > 65:
        return "B"
    elif percent > 50:
        return "D"
    else:
        return "F"
```

Test this by generating a list of random integers and computing the grades. Use `numpy` for this:

In [14]:

```python
import numpy as np
percent_scores = np.random.randint(100, size=50)
print(percent_scores)
```

```
[63 69 19 34 68 18 12 79 99 78 36 82 17 78 97 16 98 61 46 78 89 65 75  7
 41 15 62 28 70 40 26 74 98 80 43 62 46 30 76 91 13 28 51 24 82 61 97 26
 68 35]
```

In [15]:

```python
letter_grades = []
for percent in percent_scores:
    letter_grades.append(compute_grade(percent))
print(letter_grades)
```

```
['C', 'C', 'F', 'F', 'C', 'F', 'F', 'C', 'A', 'C', 'F', 'A', 'F', 'C', 'A', 'F', 'A', 'C', 'F', 'C', 'A', 'C', 'C', 'F', 'F', 'F', 'C', 'F', 'C', 'F', 'F', 'C', 'A', 'C', 'F', 'C', 'F', 'F', 'C', 'A', 'F', 'F', 'D',
'F', 'A', 'C', 'A', 'F', 'C', 'F']
```

Unexpected result: No B grades

How do we debug? Try to find a minimal example where the code does the wrong thing (returns an incorrect result).

Copy code, so we can debug in one cell:

In [19]:

```python
def compute_grade(percent):
    if percent > 80:
        return "A"
    elif percent > 55:
        return "C"
    elif percent > 65:
        return "B"
    elif percent > 50:
        return "D"
    else:
        return "F"

compute_grade(75)
```

Out[19]:

```
'C'
```

Definitely wrong, and also simplest possible example - one input, one output, not the right output.

Set breakpoint and run.

- First see "if percent > 80" with correct value for percent.
- Step once, at "elif percent > 55"
- Step again, inside this clause of if statement. Not where I expect to be.

Why? Comparisons are in the wrong order.

Stop execution and fix code

In [20]:

```python
def compute_grade(percent):
    if percent > 80:
        return "A"
    elif percent > 65:
        return "B"
    elif percent > 55:
        return "C"
    elif percent > 50:
        return "D"
    else:
        return "F"

compute_grade(75)
```

Out[20]:

```
'B'
```

Now I have fixed behaviour on my MFE. Go back and test bigger test case

In [21]:

```python
letter_grades = []
for percent in percent_scores:
    letter_grades.append(compute_grade(percent))
print(letter_grades)
```

```
['C', 'B', 'F', 'F', 'B', 'F', 'F', 'B', 'A', 'B', 'F', 'A', 'F', 'B', 'A', 'F', 'A', 'C', 'F', 'B', 'A', 'C', 'B', 'F', 'F', 'F', 'C', 'F', 'B', 'F', 'F', 'B', 'A', 'B', 'F', 'C', 'F', 'F', 'B', 'A', 'F', 'F', 'D',
'F', 'A', 'C', 'A', 'F', 'B', 'F']
```

Now results look plausible.

Next week, we're going to move on to using some more powerful libraries. But first, take a moment to review some features on python at the command line.

One important concept is that python modules and python scripts are two different things.

If we look at `debug_test.py`, this is a python script, since it contains code outside of its functions:

Try to use it as a module:

In [22]:

```python
import debug_test
debug_test.my_mean([2,2,2])
```

```
3.5
```

Out[22]:

```
2.0
```

In [23]:

```python
debug_test.my_list
```

Out[23]:

```
[1, 2, 3, 4, 5, 6]
```

We've gotten too much, including the data in the test case.

Proper way to do this is to use an if statement between module functions and rest of code:

Add:

```python
if __name__ == '__main__':
    my_list = [1, 2, 3, 4, 5, 6]
    print(my_mean(my_list))
```

This now checks if the code is being run at the command line, or not. And if it is, runs the code as usual. Otherwise, just imports the functions in the module.

At command line:

```
(CMSC6950_venv) [math-scott02:CMSC6950/Fall2025/Notes] smaclachlan% python debug_test.py
3.5
```

In [24]:

```python
import debug_test
```